Lab 2 - Cloud Data, Stat 214, Spring 2025

March 21, 2026

1 Introduction

In this project, we will be working with image classification data from NASA's Multi-angle Imaging SpectroRadiometer (MISR) sensor. As the name suggests, MISR takes satellite images from different camera angles. This is useful for a key goal of MISR: cloud detection in the Arctic. Typically, we can distinguish cloudy and clear skies based on their color: A white-colored area is deemed cloudy and darker areas are deemed clear. But since snow- and ice-covered surfaces have similar light scattering properties to cloud particles, that distinction is no longer a good basis for cloud detection.

Another way we often distinguish clouds and non-clouds is their altitude: Since clouds are at a higher altitude, they tend to have lower temperatures. But in the icy cold Arctic, this is also not a practical means of distinction. However, one useful advent of clouds' higher altitude is that the light they emit to the MISR cameras arrives at different angles compared to the light from snow- and ice-covered surfaces. We use the radiances (i.e., brightnesses) from different cameras to compare cloudy and clear images.

This angular insight is a major start to solving the problem, which has important implications outside of satellite imagery: Clouds play a critical role in channeling or mitigating the flow of electromagnetic radiation in the Arctic. Depending on the trends in cloudcover, it is possible for more heat and light from the sun to be conducted to the surface, melt sea ice, and unleash methane from permafrost that could significantly exacerbate climate change. Moreover, in their 2008 paper, Shi et al. write that modern global climate models predict that "the strongest dependences of surface air temperatures on increasing atmospheric carbon dioxide levels will occur in the Arctic". In other words, the Arctic is where the impact of cloudcover on carbon dioxide levels is most critical, which makes it troubling that it is where we can map cloudcover the least.

As Shi et al. write, if we don't accurately map out the location of Arctic clouds, then we will be in the dark about patterns in electromagnetic radiation in the Arctic and in turn trends in Arctic warming. Also, if the public and lawmakers understood climate trends more accurately, they might be more likely to recognize the climate threat and effect environmental change.

Broader impacts aside, in this project specifically, we are working with 164 MISR images of the Arctic with about 115,000 pixels each. For each pixel, we are given its planar coordinates, its radiances from five of MISR's nine cameras, and the values of three synthetic features constructed by Shi et al: The average linear correlation (CORR) between radiances at several angles, the standard deviation (SD) between groups of red radiation measurements from MISR's An camera (i.e., the one that faces directly downwards), and the normalized difference angular index (NDAI), which is the ratio between the difference and sum of two radiance angle measurements. For 3 of the images, we also have expert labels for whether each pixel is cloudy or clear, but some of these pixels are left unclassified due to lack of confidence.

Our goal is to use the data to make cloud prediction algorithms, as Shi et al. did in their paper. We start by performing an exploratory data analysis (EDA) to get a broad idea of the relationships between the features and how they compare between cloudy and clear pixels. Then, we perform feature engineering, ranking the existing features and using their importance to construct new ones: this is our attempt at the expert-informed way Shi et al. constructed NDAI, SD, and CORR. After this, we do transfer learning, wherein we train an autoencoder on the unlabeled images and finetune it on the labeled ones. This allows us to select a set of optimal, latent variables that we will use to make our prediction models.

In that vein, in the modeling section, we train and test four classes of models: (1) Random Forest, (2) Logistic Regression, (3) Linear Discriminant Analysis, and (4) LightGBM. We assess how accurately the

models at cloud detection on the labeled data, what insights we can learn the model output, what patterns arise in our classification errors, how stable our models are to data perturbations, and if they seem to make reasonable predictions on the unlabeled data. Finally, we conclude by reflecting on our analysis and model choices, sharing insights we gained on the Arctic cloud detection challenge.

2 EDA

2.1 Data Cleaning

As alluded to in the introduction, the data is 164 MISR images, represented as 164 .npz files in numpy's float64 format. We found it very computationally costly to work with these files in their raw form, so in our analysis pipeline, we converted them to float32 format and storing them in a new directory `image_data_float32`: the code for this is in `run.sh`. We also modified the `make_data` function in `data.py` to read the float32 files and to read the files in a more vectorized way.

By default, `make_data` gives us a list of numpy arrays (images_long) for each image and another list of patches, wherein each pixel is padded to form a 9x9 square with 8 features: this becomes very important in transfer learning. But in the EDA, since we are just interested in the data and its relationships, we added an optional parameter to `make_data` that allows us both to keep the expert labels in the labeled images, which are removed by default, and also to avoid computing the patches, as they are generated from the images.

Regarding cleaning, we checked for potential issues in `eda.ipynb`: At first, we confirmed that there were no NaN's in the images_long list, which is unsurprising since we coded the arrays as float32. But what if there were missing values imputed as zeros? Combining all the images, we found that the radiance angles, coordinates, and SD contained no zeros at all. In fact, the only features with any zeros at all were NDAI and CORR, with 8.98e-05 and 2.27e-04 percent zeros across all images, respectively. With incredibly small percentages, even if zeros truly were a stand-in for missingness, it wouldn't be worth the risk of making that assumption and potentially excluding important data.

Even so, we stratified our zero search by image: for each feature, we identified the image whose percentage of zeros was greatest among all images. Underwhelmingly, this yielded very similar results: One image had 8.67e-04 percent zeros in NDAI, another had 1.73e-03 for CORR, and all other features had no zeros at all.

One might still worry that some values fall outside the theoretical bounds of the features (e.g., CORR not being in [-1, 1]), but our feature distribution comparisons throughout the EDA find no such issue. Thus, with no apparent cause for cleaning, we decided to leave the data in float32 form.

2.2 Expert Labels

As per the instructions, for the three labeled images, we made scatterplots of the pixels on (x, y)-planes colored by whether the experts marked them as cloudy, clear, or were unsure.

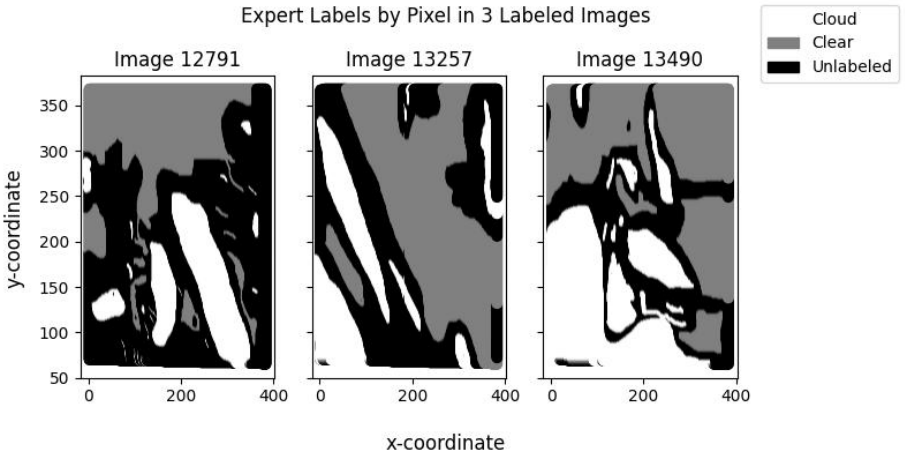


Figure 1: Expert labels by pixel for images 12791, 13257, and 13490

Note that we deliberately used the same color scheme as in Figure 4 of Shi et al., which makes this an interesting comparison. While our labeled images are 12791, 13257, and 13490, the authors used 13257, 13490, and 13723, so two out of the three are the same. Despite that, none of our plots have the same shape, although they are similar in that the gray, white, and black regions are mostly distinct. Of course, that's to be expected: surely an expert wouldn't expect a cloudy pixel to be interspersed with clear pixels and vice versa since clouds are at least visually continuous. Our plots might have different shapes since Shi et al. made plots specifically for MISR orbit blocks 20-22, whereas we weren't told which blocks yielded our data.

2.3 Radiance Angle Relationships

To get an idea of the relationships between the radiance angles, and all the main features, we made a simple correlation heatmap of the main features across all 164 images.

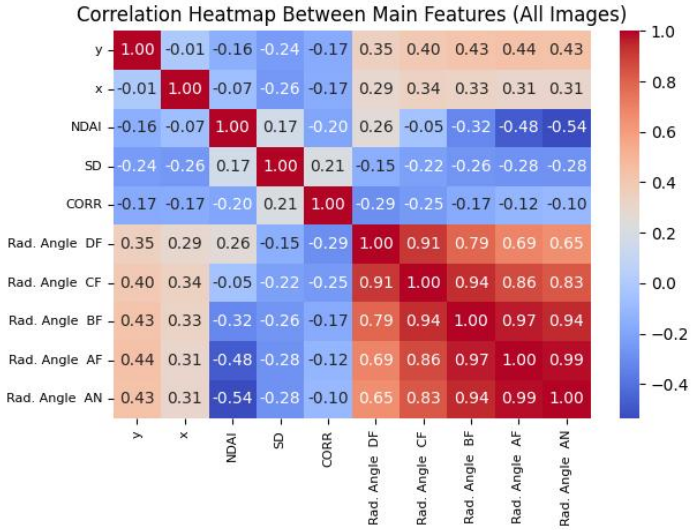


Figure 2: Correlation heatmap for all 10 main features across all images. The correlations themselves are labeled to make gradations between very high or low correlations more visible.

As we would expect, since they use the same pixels, the radiances are highly positively correlated with one another. Moreover, recall from Shi et al. that the radiance angles (in degrees) are 70.5 for Df, 60.0 for Cf, 45.6 for Bf, 26.1 for Af, and 0.0 for An, which faces directly downwards or in the nadir, whereas the other four are in the forward direction. We might expect the radiances to be closer for cameras with closer angles. In fact, in every column of the bottom quadrant of the heatmap, the correlations monotonically increase up to the diagonal and then monotonically decrease above it. That validates our intuition: The closer the angle of a camera is to our own, the more correlated its radiances are with ours.

Notably, the radiance angles are moderately negatively correlated with all three synthetic features. The only exception to this is (Radiance Angle Df, NDAI), which is likely because the formula for NDAI is

$$\text{NDAI} = \frac{\overline{Df} - \overline{An}}{\overline{Df} + \overline{An}}$$

(In reality, the formula is more complicated and takes averages of these radiance angles over a 4x4 measurement region, but this suffices to show the fundamental relationship.)

In other words, the sign of NDAI is more positive when Radiance Angle Df is larger and Radiance Angle An is smaller. That could also be why the strongest negative correlation between a radiance angle and synthetic feature is that between Radiance Angle An and NDAI.

Similarly, SD is derived from different An camera measurements. As Shi et al. write, lower values of SD are more likely to be pixels that are smooth and clear rather than cloudy. Perhaps the An camera gets

more radiance from a smooth surface rather than the closer-to-camera, more inconsistent cloud particles. If so, that would correlate low values of SD with higher values of Radiance Angle An. If that distinction is sharpest when taking images directly beneath us, then perhaps that might explain why the negative correlations between the radiance angles and SD grow weaker as the angles rise from An to Df. However, this is somewhat speculative and assumes that radiances are smaller when they have greater forward angles.

To get a better idea of the distribution of the radiance angles, we made box plots for all five. Due to runtime and memory issues, we did this only for the three labeled images.

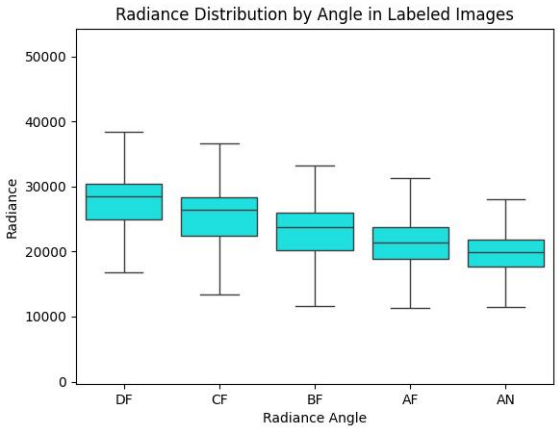


Figure 3: Distribution of radiance angle measurements for all 5 angles. Outliers were removed since they caused needless clutter and distracted from the differences between angles.

As we suggested, the distributions of the radiance angles are ordered by how far they are from the nadir direction. Here, every quartile of each radiance angle distribution monotonically decreases as we decrease the radiance angle to 0 degrees at An. This suggests the opposite of what we speculated earlier: radiances seem to be larger, not smaller, when the camera angle is tilted forward from the nadir. This isn't surprising since we were discounting the role of sunlight: To be angled away from the ground is to be angled towards the sun and to get a brighter image. This is still speculation, but it is more consistent with our findings than the extrapolation we made from the heatmap.

Next, let's compare how the radiance distributions vary based the expert label of each pixel.

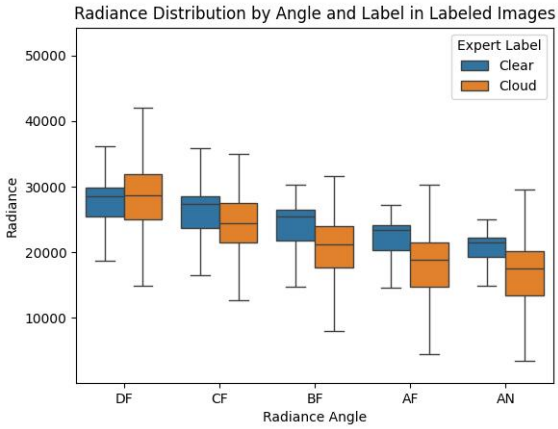


Figure 4: Distribution of radiance angle measurements for all 5 angles, stratified by whether expert label.

Firstly, notice that the three middle quartiles of each radiance distribution are consistently for pixels labeled cloudy than clear, except Radiance Angle Df. Perhaps this just means that clouds generally have lower radiances than clear areas, although we must be careful with that reasoning since cloud particles and snow- and ice-covered surfaces have similar scattering properties. More notably, the variation in the radiance angles is consistently larger for the pixels labeled cloudy than clear. This makes sense because, in the ELCM algorithm in Shi et al., an increase in CORR, which indicates more correlation between measurements and therefore less variance, is a sign that a pixel is clear. Moreover, that variation difference is sharpest for the An camera, which also makes sense because SD, which is restricted to the An camera, being low is also a predictor for a clear pixel in the ELCM algorithm. Simply put, low SD for Radiance Angle An indicates that the pixel is likelier to be clear, which is very much the case in this plot.

As for comparing differences between the synthetic features based on the expert labels, we initially tried plotting them as we did for the radiance angles, but the SD difference were far larger and drowned out the CORR and NDAI ones. Thus, we decided to use a t-test of independence to compare the pixels labeled clear and cloudy for each synthetic feature. Note that the assumption of independence is trivially unmet here: All the pixels come from the same three images, so they are very much related. But this is rather meant to be a more informal demonstration of the apparent "significance" of the differences between the clear and cloudy pixels. Computing the difference in means for each synthetic feature and the p-value from each t-test, we have the following table.

Feature	Difference in Means	T-test P-value
NDAI	0.12188	0.0
SD	559.97864	0.0
CORR	0.04678	3.99032e-143

Table 1: Difference in means (cloud - clear) among the synthetic features between pixels labeled cloudy and clear, and also include the p-value of t-tests of independence between the groups for each synthetic feature.

According to the ELCM algorithm, these results mostly make sense. Lower values of SD and NDAI are associated with clear pixels, which is consistent with the difference in means (cloud - clear) being positive for both with p-values of 0. While higher values of CORR are supposed to be associated with clear pixels, keep in mind that Shi et al. found multiple confounding variables and used SD and NDAI to adjust for them (e.g., "Smooth cloud-free terrain surfaces" could have low CORR and be classified as cloudy). Also, the difference itself is very small (0.04678) and the p-value is the only non-zero one. Of course, in a real t-test, such a minuscule p-value would always be significant, but this is a flawed t-test with dependent samples, so perhaps it is meaningful. Either way, beyond CORR, this comparison confirms our domain knowledge.

2.4 Training, Validation, and Testing Splits

At first, one might be tempted to split the labeled images randomly into 60/20/20 percent training/validation/testing splits. But when working with future data, we do not get a combination of three images but rather a single image. If our training and validation splits happen to be 90 percent image 13490, then we might fit disproportionately to its patterns relative to the other images. Thus, one approach might be to use each image as a training, validation, and testing set, respectively. However, ideally, we also want to capture as much variation as possible within each set to be as reflective of the variation in the real-world. Thus, as a compromise, our proposal is to makes stratified random splits within each of the three labeled images: Split each of them into 60/20/20 percent training/validation/testing splits and then recombine the corresponding sets from each image to make image-balanced 60/20/20 percent splits for all the labeled pixels. The choice of 60/20/20 percent is very common and something we often covered in the course and the Veridical Data Science textbook. It is distributed evenly between validation and testing but also uses most of the dataset for training, which strikes a happy medium.

In `eda.py`, we implement this approach via a function `train_val_test_split`. We use it to perform the stratified 60/20/20 split on the labeled images and write the result to `train_val_test_split.npz`.

3 Feature Engineering

3.1 Top Three Informative Features

We evaluated the existing features using both quantitative metrics and visualizations to identify the most informative predictors. Quantitatively, we calculated univariate AUC values to assess the discriminative power of each feature individually and computed Cohen's d as a standardized measure of separation.

Feature	AUC	Cohen's d	Mean per-image ROC-AUC
SD	0.9351	1.1695	0.9275
NDAI	0.8234	1.4147	0.7845
Radiance AF	0.7992	-1.1557	0.7814

Table 2: Top three original features ranked by single-feature ROC-AUC.

Table 2 shows that SD was the strongest single predictor of cloud presence, achieving the highest AUC (0.9351) and also maintaining strong performance across the three labeled images. NDAI ranked second, with a lower ROC-AUC (0.8234) but the largest absolute Cohen's d. Radiance AF was the strongest raw radiance variable, with AUC 0.7992, suggesting that the angular radiance measurements contain useful discriminatory information without additional engineering.

Visually, we examined boxplot, and density plots for the strongest feature: SD, along with selected spatial heatmaps on a labeled image.

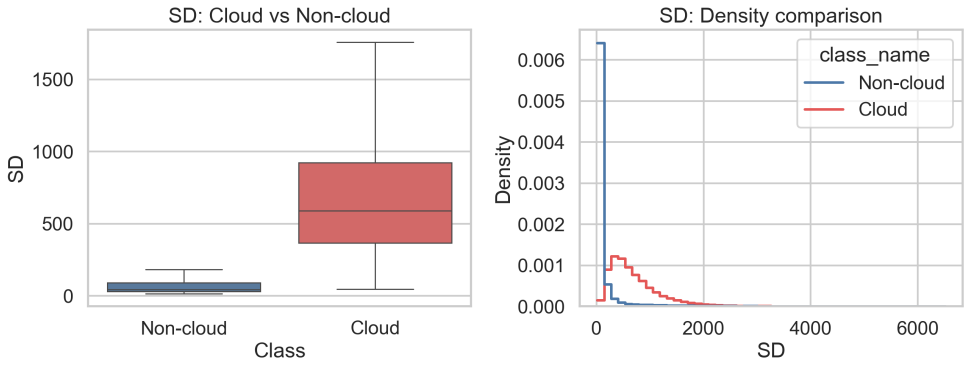


Figure 5: Boxplot and density comparison of SD for cloud and non-cloud pixels.

Figure 5 shows that SD provides strong separation between cloud and non-cloud pixels. In the boxplot, the cloud class has a higher median SD value than the non-cloud class, and its distribution is also much wider. The density plot shows the same pattern: although with some overlap remains, the cloud distribution is shifted clearly to the right, while the non-cloud distribution is concentrated at much lower SD values. This shows the strong discriminatory power of SD. The separation comes from a broad overall shift in the distribution and the limited overlap suggests that even a simple threshold on SD could classify many pixels reasonably well.

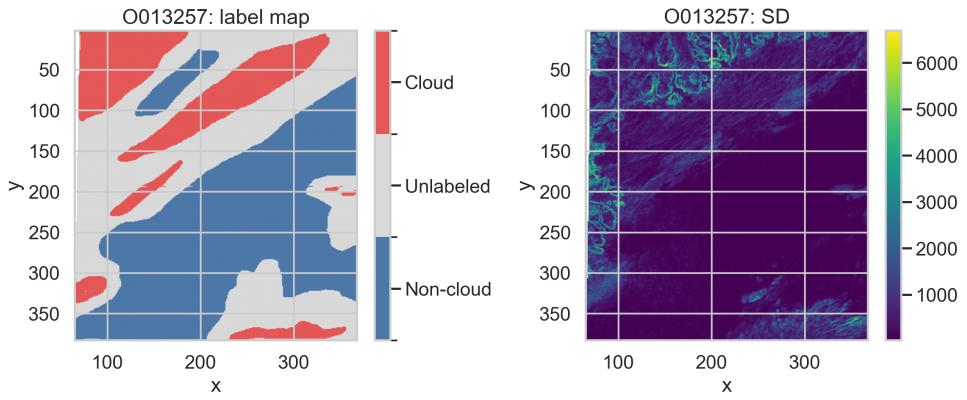


Figure 6: Expert label map and SD heatmap for image `0013257`.

Figure 6 visualizes high-SD regions tend to overlap with cloud-labeled areas, whereas large non-cloud regions are mostly characterized by low SD values. In this graph, we can observe high-SD values are concentrates at top right and the bottom, which aligns with the area of cloud in label map, indicating SD is highly informative for cloud detection.

To engineer new features, we moved beyond single-pixel measurements and incorporated local spatial context. This was motivated by the fact that cloud patterns are spatially structured: neighboring pixels often belong to the same region and share similar texture, variability, and boundary characteristics. As a result, a feature computed from a small local neighborhood may be more informative than the raw value at a single pixel.

3.2 Patch-based feature engineering

In the labeled images, cloud and non-cloud pixels formed coherent spatial regions with visible boundaries and local texture. This suggested that cloud detection is not purely a single-pixel classification problem. A pixel's local neighborhood may contain useful information about whether that pixel belongs to a cloud region, especially near boundaries or in textured areas.

This observation motivated us to construct patch-based features that incorporate local spatial context. We chose a 3×3 neighborhood around each pixel. For each of the three domain-relevant variables 'NDAI', 'SD', and 'CORR', we computed the local mean and local standard deviation within the 3×3 window. These engineered variables were intended to capture two complementary forms of neighborhood information: the average local level of the feature and the amount of local variability.

Feature	Type	AUC
SD	Original	0.9351
SD_mean_3x3	Engineered	0.9363
NDAI	Original	0.8234
NDAI_std_3x3	Engineered	0.9274
CORR	Original	0.5237
CORR_mean_3x3	Engineered	0.5276
CORR_std_3x3	Engineered	0.5124

Table 3: Comparison of original and patch-based engineered features using single-feature ROC-AUC.

Table 3 shows that patch-based feature engineering improved some variables. The largest gain came from `NDAI_std_3x3`, whose ROC-AUC increased substantially relative to raw NDAI, suggesting that local variability carries important discriminatory information. `SD_mean_3x3` also slightly improved upon raw SD. In contrast, the CORR-based engineered features remained weak.

4 Transfer Learning

4.1 Motivation

Because we only have a small amount of labeled data but much more unlabeled data, training a supervised model directly may lead to overfitting and unstable results. To address this, we use transfer learning with an autoencoder.

We first pretrain the autoencoder on all unlabeled data, allowing it to learn useful local structure through reconstruction. We then fine-tune the pretrained model on the three labeled target images, but without using the labels at this stage. The labels are reserved for the later classification task.

This approach provides a better initialization than training from scratch and helps the learned representation adapt to the target images. As a result, the encoder's latent features are expected to be more useful for downstream classification.

4.2 Part A: Transfer Learning with Autoencoder

4.2.1 Overview of Our Pipeline

Our pipeline has two stages. First, we pretrain an autoencoder on many unlabeled images. Second, we fine-tune it on the three target images using the same reconstruction objective. In this stage, we first use cross-validation to choose the learning rate and number of epochs, then run final fine-tuning on all three images.

The result is a fine-tuned encoder, not a classifier. It maps each spatial patch to a latent vector, which is later used as input for the classification models.

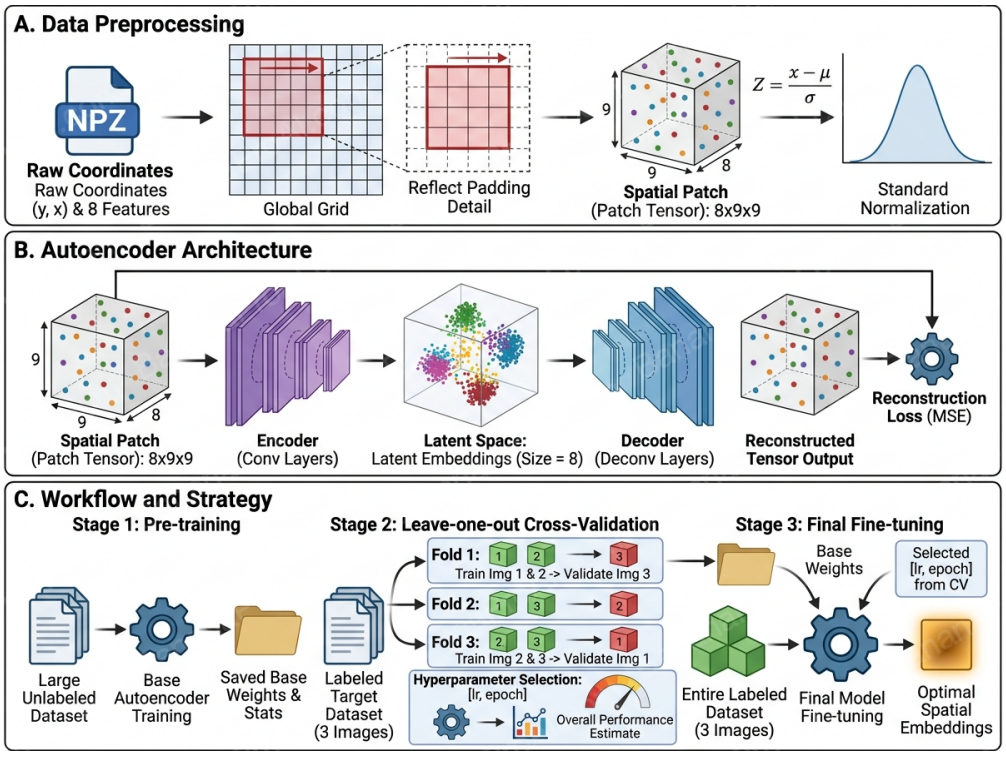


Figure 7: Overview of the transfer learning pipeline.

4.2.2 Data Preparation

Each `.npz` file is loaded as a pixel-level table and converted into an 8-channel spatial tensor in a shared coordinate frame. We then apply channel-wise normalization using statistics from pretraining and extract

9×9 patches with reflect padding, so each sample has shape $8 \times 9 \times 9$. This gives the model local spatial context.

4.2.3 Pretraining on Unlabeled Images

In the pretraining stage, we use 161 unlabeled images. The split is done at the image level: 80% of the images are used for training and 20% are used for validation. We choose image-level splitting because patches from the same image are strongly related. If we split at the patch level, training and validation would contain very similar patches, and the validation loss would look better than it really is.

After normalization and patch extraction, all training patches are combined into one training set, and all validation patches are combined into one validation set. The autoencoder is then trained to reconstruct the input patches. The best checkpoints are selected by validation loss, and the normalization statistics are also saved for later use.

In our experiments, the pretraining learning rate is 10^{-3}, the batch size is 1024, and the maximum number of epochs is 20. This stage gives the encoder a reasonable representation before we move to the small target dataset.

4.2.4 Fine-Tuning on the Three Labeled Images

After pretraining, we load the pretrained checkpoint and the saved normalization statistics. Then we fine-tune the autoencoder on the three labeled target images. Even though these images have expert labels, the fine-tuning loss is still only reconstruction loss. The purpose here is not to train a classifier yet, but to make the pretrained representation fit the target images better.

In our final setup, all model parameters are trainable during fine-tuning. We do not freeze the encoder or decoder. Since the target images are the final domain we care about, we let the full model adjust to them.We use two fine-tuning steps.

Cross-validation fine-tuning. First, we do leave-one-image-out cross-validation on the three labeled images. In each fold, two images are used for training and one image is used for validation. This gives three folds in total. Since the dataset is very small, this step is useful for checking whether the model behaves similarly across different target images. For each fold, we record the best validation loss, and we also report the mean and standard deviation over the three folds. We choose the hyperpamameters learning rate and the number of epochs based on the validation loss.

Final fine-tuning. After cross-validation, we train again on all three labeled images together. This run is used to produce the final checkpoint for embedding extraction. At this point, we are no longer trying to compare folds or choose settings. We just want to use all target images to get the final encoder.

For fine-tuning, we use a smaller learning rate, 10^{-4}. The maximum number of epochs is 20 for the cross-validation runs and 5 for the final fine-tuning run. Since the model already starts from pretrained weights, a smaller learning rate is enough for adaptation.

4.3 Part B: Autoencoder Architecture and Embedding Design

We followed the course-template autoencoder pipeline and used the learned latent representations as transferable features for downstream classification. The model takes a 9×9 patch with 8 input channels, flattens it, maps it into a low-dimensional latent space with an MLP encoder, and reconstructs the patch with a decoder. For reproducibility, stage-specific settings were stored in `pretrain.yaml`, `finetune_cv.yaml`, and `finetune_final.yaml`. Our contribution in Part B was to modify the activation function, compare the resulting embeddings with the baseline template, and export the final latent features for downstream modeling.

4.3.1 Baseline and Modified Autoencoder

The baseline model is the template autoencoder with ReLU activations in both encoder and decoder. The encoder maps the flattened patch through hidden layers of sizes 128 and 64 before projecting to the latent space, and the decoder mirrors this structure back to the original patch dimension.

Our modified model keeps the same encoder–decoder structure and changes only the activation function from

ReLU to LeakyReLU. Specifically, all hidden-layer activations were replaced by `LeakyReLU(negative_slope=0.1)`. The motivation was to reduce dead-ReLU behavior while keeping the overall model capacity unchanged. Because no other architectural changes were introduced, the comparison isolates the effect of activation choice.

4.3.2 Embedding Evaluation and Selection Criteria

We evaluated the embeddings using three criteria: qualitative visualization, latent-dimension comparison, and a quick downstream probe. Figures 8 and 9 compare the baseline and modified embeddings.

For the baseline model, PCA shows substantial overlap between cloud and non-cloud pixels, while t-SNE reveals clearer local structure and partial class separation. Thus, the baseline embedding captures useful nonlinear structure even though the first two principal components do not cleanly separate the classes.

For the modified LeakyReLU model, the PCA embedding is more compact, with fewer extreme outliers and a more concentrated point cloud. The t-SNE view still shows nonlinear local organization and somewhat clearer cloud/no-cloud grouping in several regions. Overall, the modified model appears to preserve the useful structure of the baseline embedding while reducing some instability in PCA space.

Latent-dimension comparison determined whether a compact representation was sufficient, the quick probe measured immediate downstream usefulness, and PCA/t-SNE served as a qualitative sanity check on embedding geometry.

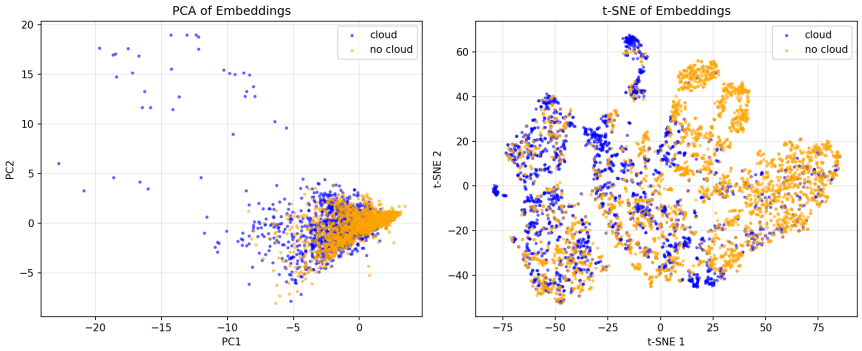


Figure 8: PCA and t-SNE visualizations of the baseline autoencoder embeddings. PCA shows strong overlap between cloud and non-cloud pixels, while t-SNE reveals clearer local clustering and partial separation.

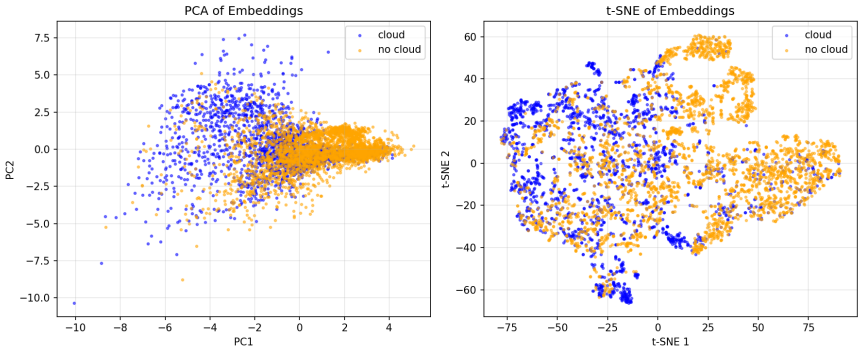


Figure 9: PCA and t-SNE visualizations of the modified LeakyReLU autoencoder embeddings. Compared with the baseline, the PCA representation is more compact and exhibits fewer extreme outliers, while the t-SNE view preserves meaningful local structure with slightly clearer class organization in several regions.

4.3.3 Final Checkpoint and Latent Dimension

The final feature-extraction pipeline followed a pretraining–fine-tuning strategy. In the logged fine-tuning run, normalization statistics were loaded from `checkpoints/pretrain/norm_stats.npz`. The baseline variant was initialized from `pretrain-epoch=014-val_loss=0.1063.ckpt`, and the modified LeakyReLU variant from `pretrain-epoch=016-val_loss=0.1065.ckpt`. The run log further indicates that the final fine-tuning stage used 345,005 training patches and saved its output under `checkpoints/finetune/final`.

The final fine-tuning log reports 92,488 trainable parameters out of 184,336 total parameters (50.17%). The trainable parameter names correspond to decoder layers, suggesting a partially frozen setup that preserves part of the learned encoder representation while allowing decoder-side adaptation during reconstruction-based fine-tuning.

For both variants, the retained latent dimension used in downstream analysis is 8, as indicated by the exported columns `ae0`, ..., `ae7`. Relative to the baseline, the LeakyReLU model yields comparable quick-probe mean accuracy, slightly higher quick-probe mean ROC AUC, and a more compact PCA embedding at the same latent dimension. These results support using the LeakyReLU variant in downstream experiments.

4.3.4 Extracted Latent Features for Downstream Classification

After training, we extracted latent features for the three labeled images and saved them as `image1_ae.csv`, ..., `image3_ae.csv`. Each file contains pixel coordinates `(y, x)` together with latent variables `ae0`, ..., `ae7`. This preserves one-to-one alignment between each pixel and its embedding and allows the latent representation to be merged directly with the labeled data by spatial coordinates.

These exported embeddings form the interface between transfer learning and predictive modeling. In Part 3, they are used to construct both latent-only and raw-plus-latent feature sets. Because the modified LeakyReLU embedding is more compact in PCA space and achieves a quantitative profile comparable to the baseline, with slightly higher mean ROC AUC in the quick-probe comparison, it was selected as the primary embedding source for the final downstream model comparison.

5 Modeling

5.1 Part A: Random Forest

5.1.1 Feature Sets and Model Setup

For cloud classification, we tested Random Forest on three feature sets: original features, latent features, and their combination. The latent features were the 8-dimensional vectors extracted from the fine-tuned encoder. This was used to evaluate whether transfer learning improves downstream classification.

We used Random Forest because it can capture nonlinear patterns and feature interactions. To account for spatial dependence, we used leave-one-image-out cross-validation, holding out one full labeled image in each fold.

5.1.2 Cross-Validation Results and Model Selection

We first compared the three feature settings under the same Random Forest model. Table 4 summarizes the overall leave-one-image-out results.

Table 4: Leave-one-image-out performance of the Random Forest classifier under three feature settings.

Feature set	Mean ROC-AUC	Mean Accuracy	Mean F1
handcrafted	0.8711	0.7745	0.7939
ae_only	0.9238	0.8334	0.7934
combined	0.9035	0.7771	0.7914

Among the three settings, the best model was Random Forest with `ae_only` features, achieving the highest mean ROC-AUC of 0.924. It was also the most consistent across folds, suggesting that the autoencoder latent features transfer better across different scenes.

5.1.3 ROC Curves and Fold-wise Performance

To examine the selected model in more detail, we plotted the ROC curves for the three leave-one-image-out folds.

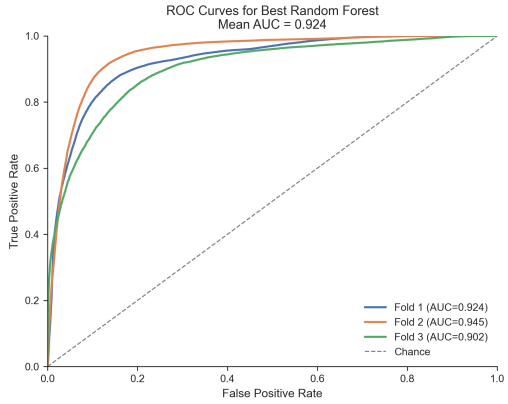


Figure 10: ROC curves for the Random Forest using only latent features, with consistent performance across the three held-out images and mean AUC of 0.924.

Figure 10 shows that all three curves stay well above the diagonal baseline, and the fold-to-fold variation is not large. Fold 2 gives the strongest result, while Fold 3 is the weakest, but its AUC is still above 0.90. Overall, this plot suggests that the selected model has fairly consistent ranking performance on unseen labeled images.

5.1.4 Confusion Matrices

We also examined the confusion matrix for each fold to better understand the classification errors.

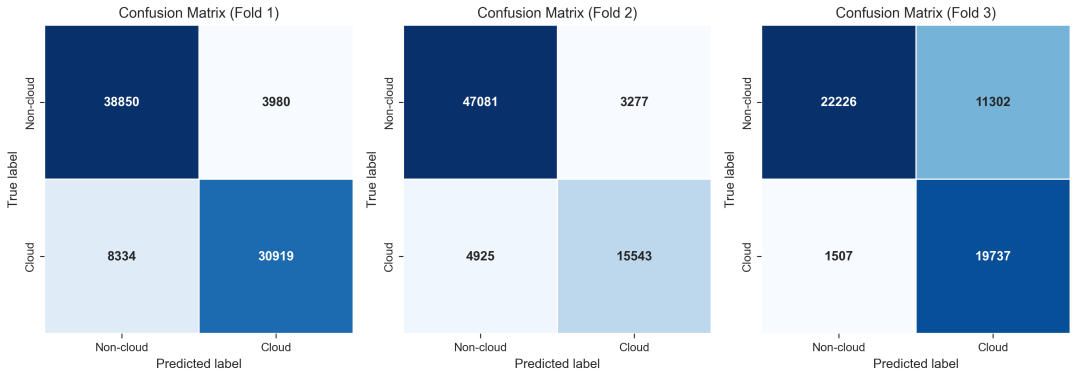


Figure 11: Confusion matrices for the three leave-one-image-out folds of the selected Random Forest model. Fold 3 shows more confusion between cloud and non-cloud pixels than the other two folds.

Figure 11 gives a more detailed view of the errors. In Fold 1 and Fold 2, the classifier performs strongly on both cloud and non-cloud pixels. Fold 3 is clearly more difficult. In that fold, the model still detects many cloud pixels correctly, but it produces more false positives for the non-cloud class. This pattern is consistent with the weaker ROC performance in Fold 3 and suggests that one labeled image is harder to separate than the other two.

5.1.5 Feature Importance

To understand which latent dimensions were most useful, we examined feature importance for the selected model using both MDI importance and permutation importance.

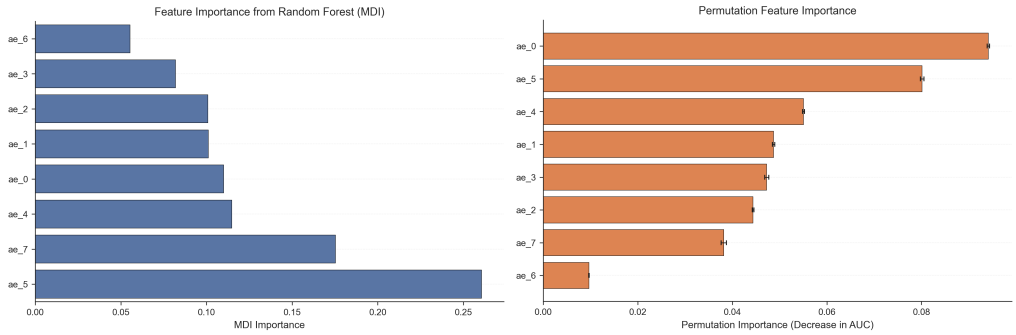


Figure 12: Feature importance for the selected `ae_only` Random Forest model. The most important latent dimensions are `ae_0` and `ae_5`, with `ae_4` and `ae_7` also contributing.

Figure 12 shows that the two rankings are not identical, but they are broadly consistent. In particular, `ae_0` and `ae_5` appear among the most important latent features in both plots, while `ae_4` and `ae_7` also make relatively strong contributions. This suggests that the learned latent space is informative, but not all dimensions are equally important for cloud prediction.

5.1.6 Post-hoc Error Analysis

To better understand the model errors, we looked at the misclassification pattern across images and the prediction confidence distribution.

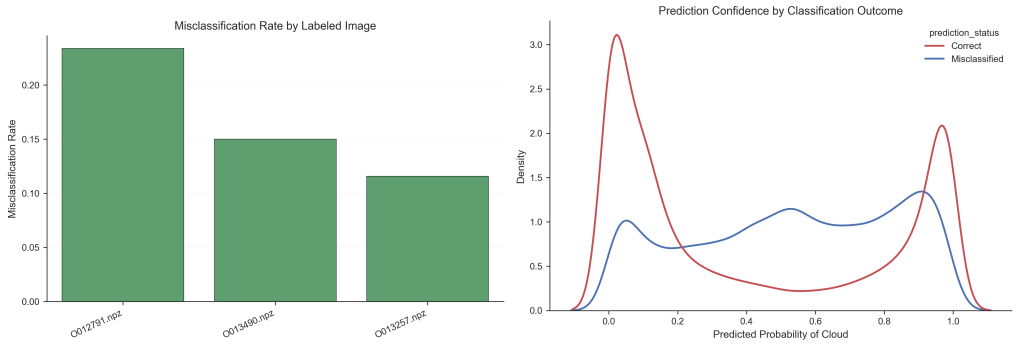


Figure 13: Post-hoc error analysis for the selected Random Forest model. Errors vary by image and are more common when predicted cloud probabilities are less certain.

Figure 13 shows that image `0012791.npz` has the highest misclassification rate, while `0013257.npz` has the lowest. This agrees with the cross-validation results and again suggests that some scenes are more difficult than others. The confidence plot shows another clear pattern: correct predictions are concentrated near probabilities close to 0 or 1, while misclassified points appear more often in the middle range. This means that many errors happen in less certain cases rather than in highly confident predictions.

5.1.7 Stability and Generalization Checks

We assessed model stability by repeating grouped cross-validation with different random seeds and by adding small Gaussian noise to the latent features. Similar metrics across seeds and small prediction changes under perturbation indicate that the selected Random Forest is both accurate and robust.

5.1.8 Summary

Overall, the Random Forest results show that transfer learning was effective. Using only the fine-tuned autoencoder latent features gave the best ROC-AUC and the most stable performance across images, suggesting that the learned representation captures cloud-related structure better than the handcrafted features.

5.2 Part B: Logistic Regression

5.2.1 Input Features and Data Construction

The logistic-regression experiments use the three labeled images from the supervised portion of the lab. For each labeled pixel, we considered three feature sets:

1. **Raw features only**, using the original non-AE variables;
2. **Latent features only**, using the autoencoder outputs `ae0`, ..., `ae7`;
3. **Raw + latent features**, obtained by merging both sources on pixel coordinates `(y, x)`.

The latent features come from the transfer-learning pipeline in the previous section. Because the exported embeddings preserve one-to-one alignment with pixel coordinates, they can be merged directly with the labeled raw-feature table. This setup lets us test both whether the learned representation is useful on its own and whether it adds signal beyond the original predictors.

5.2.2 Training/Test Design and Cross-Validation Strategy

We evaluated generalization with an image-level leave-one-image-out design. In each fold, two labeled images were used for training and the remaining image was held out for testing. This was repeated three times so that each labeled image served once as the test image.

This design is more appropriate than a random pixel-level split because nearby pixels within the same image are highly correlated. Holding out a full image therefore gives a more realistic estimate of performance on unseen data.

5.2.3 Logistic Regression Specification and Assumption Checks

For each feature set, logistic regression models cloud probability as

$$\Pr(Y = 1 \mid X = x) = \frac{1}{1 + \exp\big(-(\beta_0 + x^\top \beta)\big)}.$$

Thus, the classifier assumes that the log-odds of cloud presence are approximately linear in the supplied predictors. We evaluated each model using ROC AUC, balanced accuracy, and F1 on the held-out image in each fold.

This assumption appears more reasonable for the raw and raw-plus-latent feature sets than for the latent-only feature set. In particular, latent-only models attain moderate AUC but weaker balanced accuracy and F1, indicating useful ranking information but poorer thresholded classification. By contrast, the raw-feature models perform strongly under a linear decision rule.

5.2.4 Comparison Across Feature Sets

Table 5 reports the mean leave-one-image-out performance across the three held-out images. For both baseline and modified embeddings, the **raw-only** model gives the best threshold-based performance, with mean balanced accuracy 0.791 and mean F1 0.719. The **raw-plus-latent** model gives the highest mean ROC AUC, at 0.824 for the baseline embeddings and 0.825 for the modified embeddings. The **latent-only** model is clearly weaker in balanced accuracy and F1, although its AUC remains nontrivial.

This suggests that the latent features improve ranking more than thresholded prediction. In other words, they help order pixels by cloud likelihood, but do not outperform the already strong raw baseline under the default classification threshold.

At the fold level, image2 is the easiest of the three labeled images: both raw-only and raw-plus-latent perform extremely well on this fold. By contrast, image1 and image3 are more challenging, showing meaningful image-to-image variation and reinforcing the need for image-level validation.

5.2.5 Baseline vs. Modified Latent Features

The LeakyReLU modification yields modest but consistent improvements for the **latent-only** logistic model. Relative to the baseline latent features, the modified embeddings improve mean AUC from 0.746 to 0.753, mean balanced accuracy from 0.660 to 0.665, and mean F1 from 0.565 to 0.573. These gains are small but consistent across all three aggregate metrics, suggesting that the modified embedding is slightly more useful.

Embedding source	Feature set	AUC	Balanced Acc.	F1
Baseline	Raw	0.813	0.791	0.719
Baseline	Latent	0.746	0.660	0.565
Baseline	Raw + Latent	0.824	0.766	0.687
Modified	Raw	0.813	0.791	0.719
Modified	Latent	0.753	0.665	0.573
Modified	Raw + Latent	0.825	0.765	0.687

Table 5: Mean leave-one-image-out logistic-regression performance across feature sets. The raw-only model achieves the best threshold-based metrics, while the raw-plus-latent model achieves the highest mean AUC.

Once raw features are added, the effect becomes much smaller. For the **raw-plus-latent** model, the modified embeddings increase mean AUC only slightly (from 0.824 to 0.825), while balanced accuracy decreases marginally and F1 is essentially unchanged. Thus, most predictive power in the combined model still comes from the raw features, with the latent representation offering only a modest incremental benefit.

5.2.6 Coefficient Importance and Misclassification Analysis

The standardized coefficient plots help interpret the fitted logistic models. For the raw-only model, DF is consistently positive across folds, while BF and CF are usually negative. Some coefficients are fold-dependent: for example, AF is positive in image1 and image2 but strongly negative in image3, while AN changes sign across folds. This indicates that although the raw variables are informative overall, their exact linear relationship with cloud probability depends on image-specific background conditions.

Figure 14 provides the clearest transfer-learning interpretation: the **modified raw-plus-latent model on held-out image3**. The largest coefficients are still raw variables, especially AF, AN, DF, BF, and NDAI, but several latent coordinates (`ae1`, `ae6`, `ae4`, and `ae5`) also appear in the top 10. This shows that the latent representation contributes supplementary signal even though the raw features remain dominant.

The error maps show an equally clear contrast. On **image1**, the **modified latent-only** model produces widespread structured errors along long diagonal cloud bands and nearby ambiguous regions, while the **baseline raw-only** model is much more conservative and makes more localized mistakes near boundaries. This matches the confusion-matrix summary: the latent-only model suffers from many false positives, whereas the raw-only model trades some recall for far better specificity. Figure 15 illustrates this difference directly.

A second representative case is **image3** under the **baseline raw-only** model. Even for the strongest threshold-based logistic baseline, Figure 16 shows structured errors along the elongated central cloud band and nearby boundaries. This confirms that image3 is intrinsically harder than image2 and that some residual errors reflect image difficulty rather than weak features alone.

Overall, the coefficient plots and error maps tell a consistent story: raw features dominate the linear classifier, latent features add weaker but non-negligible signal, and the main failure mode of the latent-only model is excessive false-positive prediction in spatially coherent regions.

5.2.7 Final Model Comparison

Three conclusions follow from these results. First, the original raw features are already very strong under a linear model, especially in balanced accuracy and F1. Second, latent features alone are informative but not competitive with the raw baseline, although the modified LeakyReLU embeddings improve them slightly and consistently. Third, combining raw and latent features slightly improves AUC but does not clearly improve threshold-based performance.

5.3 Part C: Linear Discriminant Analysis

5.3.1 Model Setup and Feature Sets

In this part, Linear Discriminant Analysis (LDA) is used for cloud detection. LDA is a probabilistic classifier that is suitable when the two classes can be separated by linear combinations of predictors. LDA assumes that, conditional on the class label, the predictor vector follows a multivariate Gaussian distribution with class-specific mean vectors and a common covariance matrix shared across classes. Under these assumptions,

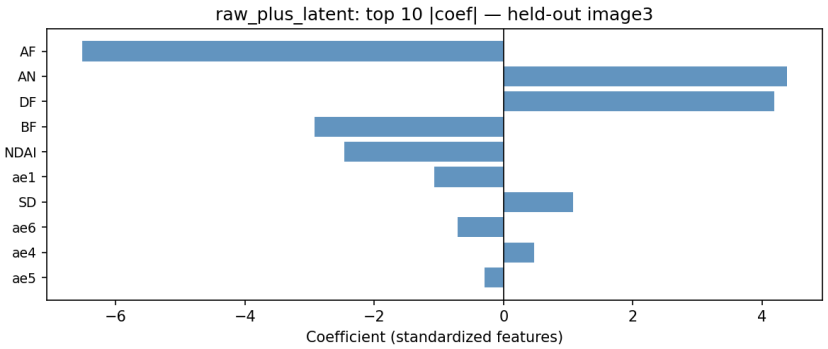
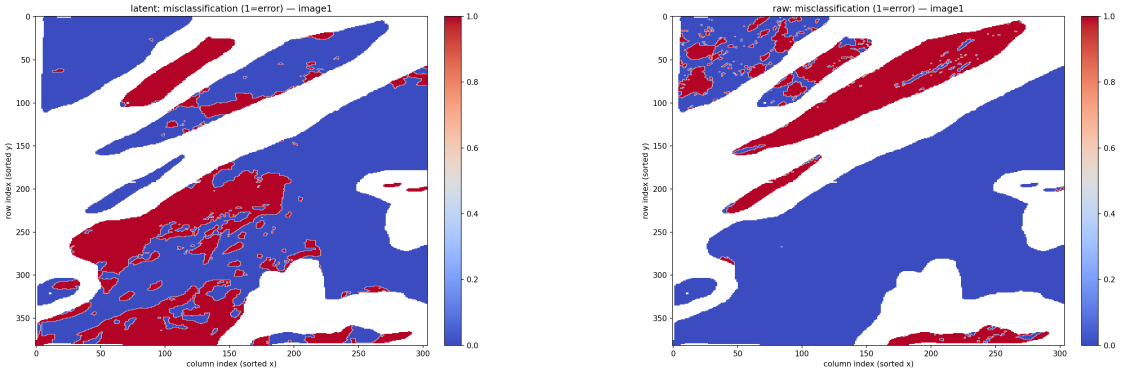


Figure 14: Top 10 standardized coefficients for the modified raw-plus-latent model on image3. Raw features dominate, with several latent features also contributing.



(a) Modified latent-only, held-out image1

(b) Baseline raw-only, held-out image1

Figure 15: Misclassification maps on image1. The latent-only model makes widespread structured errors, whereas the raw-only model is more conservative and errs mainly near boundaries.

the resulting classifier produces a linear decision boundary in feature space.

The LDA experiments were built from the same three labeled images used in the supervised modeling stage. For each labeled pixel, we considered three feature constructions: handcrafted features only, Latent features only, and handcrafted + latent features.

5.3.2 Cross-Validation Results and Model Selection

We used the same leave-one-image-out cross-validation design as in the other supervised models. Table 6 summarizes the mean leave-one-image-out performance of LDA under the three feature settings.

Feature Set	Mean AUC	Mean Accuracy	Mean F1 Score
handcrafted	0.8242	0.6426	0.8150
ae_only	0.9100	0.6677	0.7878
combined	0.8728	0.6441	0.8085

Table 6: Mean leave-one-image-out performance of LDA under different feature sets.

The best feature set by mean ROC-AUC is ae_only, with a mean ROC-AUC of 0.9100. The combined feature set improves over ‘handcrafted‘ in AUC, but it does not outperform ae_only.handcrafted achieves the highest mean accuracy (0.8150), but its AUC is notably lower than ae_only. It shows the autoencoder representation captures information that improves ranking and class separation, even when threshold-based accuracy is not maximized.

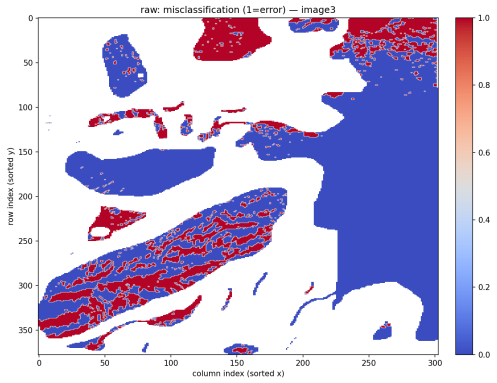


Figure 16: Misclassification map for the baseline raw-only logistic model on held-out image3. Errors cluster along the central cloud band and nearby boundaries, showing that image3 is a harder test case.

5.3.3 ROC Curves and Fold-wise Performance

In this section, we examined how LDA performed on each held-out image separately.

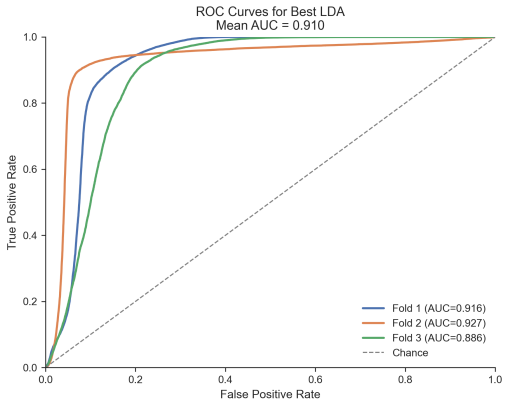


Figure 17: Fold-wise LDA performance across the three leave-one-image-out splits.

The best mean AUC is approximately 0.91 from ae_only. There is still some variation across folds. The fold-specific AUC values are approximately 0.916, 0.928, and 0.886. The second fold achieves the best performance, while the third fold is weaker, but the ROC curve remains above the chance baseline.

5.3.4 Confusion Matrices

To better understand the classification behavior of LDA, we examined the confusion matrix for each leave-one-image-out fold.

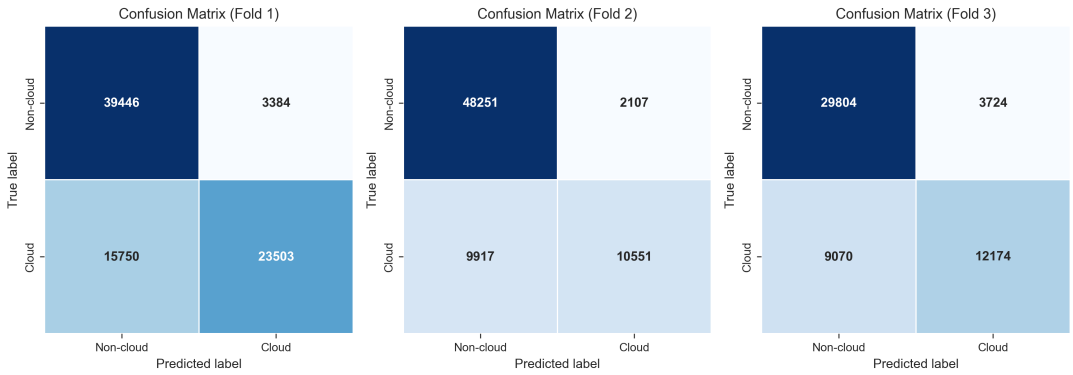


Figure 18: Confusion matrices for the three leave-one-image-out folds under the selected LDA feature setting.

In Fold 1, the model achieved 39,446 true negatives and 23,503 true positives, but still produced 15,750 false negatives, indicating that some cloudy pixels were missed. Fold 2 was the most conservative, with only 2,107 false positives but 9,917 false negatives, which explains its high precision (0.834) but relatively low recall (0.515). Fold 3 showed a more balanced but still imperfect pattern, with 29,804 true negatives, 12,174 true positives, 3,724 false positives, and 9,070 false negatives.

5.3.5 Interpretation

One advantage of LDA is that it remains relatively interpretable. Because the classifier is based on a linear discriminant rule, we can inspect the estimated discriminant coefficients to understand which predictors contribute most strongly to the separation between cloud and non-cloud pixels.

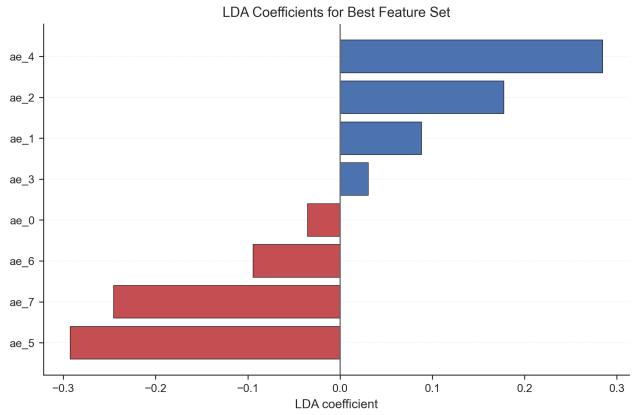


Figure 19: Estimated LDA discriminant coefficients under the selected feature setting.

The coefficient plot for the best LDA model shows the relative contribution of each autoencoder latent dimension to the linear decision boundary. Positive coefficients indicate that larger values of the corresponding latent feature push predictions toward the cloud class, while negative coefficients favor the non-cloud class. In our results, ae_4 and ae_2 had the largest positive coefficients, whereas ae_5 and ae_7 had the largest negative coefficients, suggesting that these latent directions are most influential for discrimination.

5.4 Part D: LightGBM

5.4.1 Model Description and Motivation

LightGBM is a machine learning gradient-boosting algorithm very frequently used in classification and regression, particularly for large datasets. Part of LightGBM is Gradient-based one-side sampling or GOSS, which is built on the assumption that data points with smaller gradients are less informative — they have

smaller training error, so they are not giving us information about the dataset we don't already know. This assumption is difficult to test for straightforwardly, so we find it reasonable to suppose it for now.

As for the algorithm itself, LightGBM is an ensemble of gradient-boosting models added together via gradient descent. The idea is that weak learners, when combined, will be much better at regression and classification than they are individually.

In this section, we will perform 3-fold cross-validation on the labeled images to optimize the hyperparameters of a LightGBM model for classifying the pixels as cloudy or clear. Ideally, we would have used our own `train_val_test_split` function to make the splits, which we have attempted to do, but since we generate the folds using `KFold` in sklearn anyway, this ultimately became redundant. Usually, we would use more folds (e.g., 5 or 10), but we had issues with runtime and efficiency that we mitigated by that. For simplicity, we used the default parameters of the LightGBM model for classification and put our decision threshold for cloud vs. no cloud at 0.5.

Using the latent vector from transfer learning, we made three models:

1. Model D1 trained on 8 given features
2. Model D2 trained on 8 autoencoder features from latent vector
3. Model D3 trained on 16 features from both

While we had 10 main features initially, we dropped the (x, y)-coordinates to avoid overfitting based on location in the Arctic.

To reproduce this algorithm, execute `lightgbm_mod.py` or `lightgbm_mod.ipynb`.

5.4.2 Cross-Validation Performance

Computing the average accuracy across the 3 folds for each model, we find that it is 96.108 percent for D1, 91.802 percent for D2, and 97.471 percent for D3. As we might expect, using all 16 features provides the best performance, but notably, the autoencoder features alone are worse than the given features alone. Perhaps the latent dimension we chose didn't capture as much information in the data as the original, non-coordinate features did. Then again, it's hard to tell for certain whether this 4.3 percent difference is significant in the first place.

To check its robustness, we plot the ROC curves for all three models and the AUC on the side.

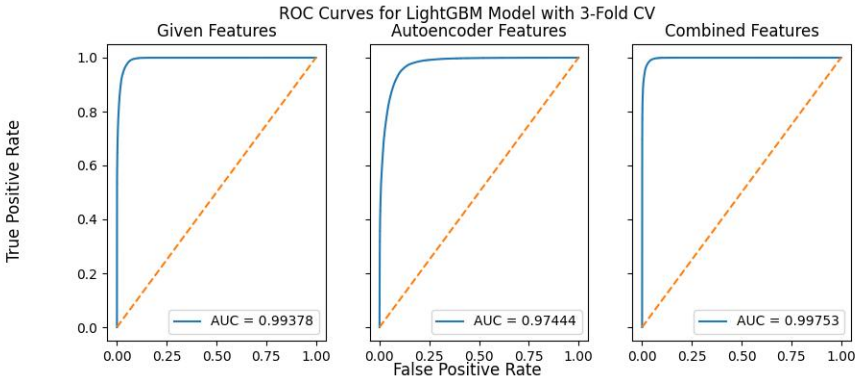


Figure 20: ROC curves for all 3 LightGBM models with AUC specified

All three models have very high AUC's, but note that their ordering matches the average CV accuracy ordering: D3 surpasses D1 and D1 surpasses D2. However, D1 and D3 are very close in AUC and trailed by a larger magnitude by D2, which reflects the trends in their average CV accuracies. Once again, the autoencoder features alone are worse than the given features alone and the two combined.

5.4.3 Feature Importance

Using the `lightgbm` package's `plot_importance` function, we plot the feature importance for each of the three models by "gain", meaning how much a feature improves accuracy from each split.

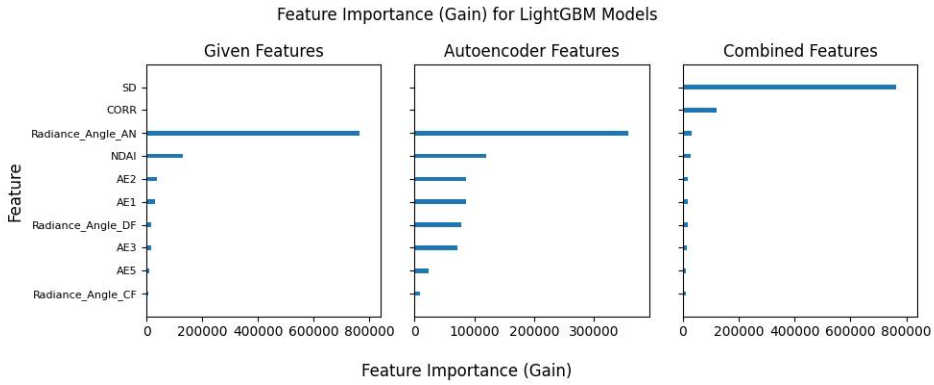


Figure 21: Feature importance via gain for all 3 LightGBM models

To avoid clutter, we didn't plot the labels on each subplot x-axis or on the bars and kept just the top 10 features. Notably, for the combined features in D3, SD was the most important feature, which is consistent with our feature engineering. For the other two, Radiance Angle An was the most important feature, which may be a glitch since Radiance Angle An is not one of the autoencoder features. However, in feature engineering, we found that Radiance Angle Af was the third most important feature. At 26.1 degrees forward, it is the closest of the five given radiance angles to Radiance Angle An at 0 degrees, so the lower angles might be more important. Then again, Radiance Angle Df surpasses Cf in feature importance for the given and autoencoder features even though Df has a much higher angle than Cf (70.5 vs 45.6 degrees). Due to lack of space and LightGBM not being the best classification model, we won't analyze this further, but our takeaway is that some critical information may be lost in the autoencoder features and, in concordance with Shi et al., SD is essential in cloud detection.

6 Conclusion

This report studies cloud classification in Arctic MISR imagery using both handcrafted features and transfer-learned latent features. An autoencoder is pretrained on unlabeled images and fine-tuned on the small labeled set to learn representations for downstream prediction.

Several classifiers are compared, including Random Forest, Logistic Regression, LDA, and LightGBM. Overall, the results show that transfer learning is effective: the Random Forest model using latent features achieved the best mean ROC-AUC, while the learned representations also improved performance or ranking ability in several other models. These findings suggest that autoencoder-based latent features provide a useful and stable representation for cloud classification when labeled data are limited.

7 Bibliography

Shi, T., Yu, B., Clothiaux, E. E., and Braverman, A. J. (2008). Daytime Arctic Cloud Detection Based on Multi-Angle Satellite Data With Case Studies. *Journal of the American Statistical Association*, 103(481), 584–597.

A Academic honesty

A.1 Statement

We certify that this report and the associated code were completed in accordance with the course academic honesty policy. All work submitted reflects our group's own understanding, analysis, implementation, and writing. Any external sources, including research papers and software tools, have been appropriately ac-

knowledged. We did not copy solutions from other students or unauthorized sources, and we did not submit work that was not meaningfully produced and verified by our group.

A.2 LLM Usage

Coding

We used large language model (LLM) tools in a limited supporting role during the coding process. Specifically, we used AI assistance to help generate figures, troubleshoot errors, and suggest revisions to parts of the code. All code was reviewed, tested, and modified by us as needed before being included in the final submission. We take full responsibility for the correctness, design, and interpretation of the final code and results.

Writing

We used LLM tools to help improve the clarity, grammar, and overall polish of the writing in this report. The underlying ideas, experimental design, analysis, and conclusions were developed by our group. AI assistance was used only for language refinement rather than for generating the substantive academic content of the report.